

```

1 #MOUNT TO SAVE AND LOAD df of built datasets
2 #to mount the personal 1612 gmail drive (IMP)
3 from google.colab import drive
4 drive.mount('/content/drive')

```

Mounted at /content/drive

```

1 df_path = '/content/drive/MyDrive/AiAudio_PR_Project/AiAudio_Dataset'
2 for_2sec_df_path = df_path+'/for_2sec' #will use it to train and select best model
3 for_2sec_rerec_df_path = df_path+'/for_2sec_rerec' #retrain, for our scenario, on the selected best model
4
5 print("Path to dataset files:", for_2sec_df_path)
6 print("Path to dataset files:", for_2sec_rerec_df_path)

```

Path to dataset files: /content/drive/MyDrive/AiAudio_PR_Project/AiAudio_Dataset/for_2sec
 Path to dataset files: /content/drive/MyDrive/AiAudio_PR_Project/AiAudio_Dataset/for_2sec_rerec

1 Start coding or [generate](#) with AI.

✓ Getting data

```

1 import kagglehub
2
3 # Download latest version
4 path = kagglehub.dataset_download("mohammedabdeldayem/the-fake-or-real-dataset")
5

```

Downloading from <https://www.kaggle.com/api/v1/datasets/download/mohammedabdeldayem/the-fake-or-real-dataset?data>
 100%|██████████| 16.0G/16.0G [03:36<00:00, 79.4MB/s]Extracting files...

```

1 import os
2 cnn_path = os.path.join(path, "for-2sec/for-2seconds")
3 for_2sec_path=os.path.join(path, "for-2sec/for-2seconds/training")
4 for_2sec_test_path=os.path.join(path, "for-2sec/for-2seconds/testing")
5 for_2sec_valid_path=os.path.join(path, "for-2sec/for-2seconds/validation")
6
7
8
9 print("Path to dataset files:", path)
10 print("Path to dataset files:", for_2sec_path)

```

Path to dataset files: /root/.cache/kagglehub/datasets/mohammedabdeldayem/the-fake-or-real-dataset/versions/2/
 Path to dataset files: /root/.cache/kagglehub/datasets/mohammedabdeldayem/the-fake-or-real-dataset/versions/2/for

```

1 #RERECORDED FoR DATA, FOR INFERENCE
2 for_rerec_path_real = os.path.join(path, "for-rerec/for-rerecorded/validation/real")
3 for_rerec_path_fake = os.path.join(path, "for-rerec/for-rerecorded/validation/fake")
4
5 print("Path to rerec real files:", for_rerec_path_real)
6 print("Path to rerec fake files:", for_rerec_path_fake)

```

Path to rerec real files: /root/.cache/kagglehub/datasets/mohammedabdeldayem/the-fake-or-real-dataset/versions/2/
 Path to rerec fake files: /root/.cache/kagglehub/datasets/mohammedabdeldayem/the-fake-or-real-dataset/versions/2/

✓ Feature Extraction

✓ Function

```

1 import os
2 import librosa
3 import numpy as np
4 import pandas as pd
5
6 def extract_features(file_path):
7     y, sr = librosa.load(file_path, sr=16000)
8
9
10     mfccs = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
11

```

```

12     delta_mfccs = librosa.feature.delta(mfccs)
13
14     spec_cent = librosa.feature.spectral_centroid(y=y, sr=sr)
15
16     spec_bw = librosa.feature.spectral_bandwidth(y=y, sr=sr)
17
18     chroma = librosa.feature.chroma_stft(y=y, sr=sr)
19
20     zcr = librosa.feature.zero_crossing_rate(y)
21
22     rms = librosa.feature.rms(y=y)
23
24
25     features = []
26     for feature_set in [mfccs, delta_mfccs, spec_cent, spec_bw, chroma, zcr, rms]:
27         features.extend(np.mean(feature_set, axis=1))
28         features.extend(np.std(feature_set, axis=1))
29
30     return features
31
32
33 def build_dataset(base_dir):
34     data = []
35     for label_dir in ['real', 'fake']:
36         folder = os.path.join(base_dir, label_dir)
37         label = 0 if label_dir == 'real' else 1
38         for fname in os.listdir(folder):
39             fpath = os.path.join(folder, fname)
40             try:
41                 feats = extract_features(fpath)
42                 data.append([fpath] + feats + [label])
43             except Exception as e:
44                 print(f"Error with {fpath}: {e}")
45     return data
46
47
48 feature_names = [f'mfcc{i}' for i in range(13)] + \
49                 [f'delta_mfcc{i}' for i in range(13)] + \
50                 ['spec_cent', 'spec_bw'] + \
51                 [f'chroma{i}' for i in range(12)] + \
52                 ['zcr', 'rms']
53
54 feature_names = [f'{f}_{stat}' for f in feature_names for stat in ['mean', 'std']]
55 df_cols = ['filename'] + feature_names + ['label']
56
57
58

```

for_2sec Dataset

No need to run Below again, just load pre-save(unprocessed)

The training dataset

```

1 dataset = build_dataset(for_2sec_path)
2 df = pd.DataFrame(dataset, columns=df_cols)

```

```

1 df.head()

```

```

1 df.tail()

```

```

1 df.shape

```

```

1 print(for_2sec_df_path+"/training_df.pkl")

```

```

1 df.to_pickle(f"{for_2sec_df_path}/training_df.pkl")

```

The testing set

```

1 dataset_test = build_dataset(for_2sec_test_path)
2 df_test = pd.DataFrame(dataset_test, columns=df_cols)

```

```
1 df_test.head()
```

```
1 df_test.tail()
```

```
1 df_test.shape
```

```
1 df_test.to_pickle(f"{for_2sec_df_path}/testing_df.pkl")
```

- ✓ Loading Dataset

for_2sec

```
1 df = pd.read_pickle(f"{for_2sec_df_path}/training_df.pkl")
2 df_test = pd.read_pickle(f"{for_2sec_df_path}/testing_df.pkl")
```

```
1 print(df.columns.tolist())
```

```
[ 'filename', 'mfcc0_mean', 'mfcc0_std', 'mfcc1_mean', 'mfcc1_std', 'mfcc2_mean', 'mfcc2_std', 'mfcc3_mean', 'mfcc3_std',
```

- Preprocessing

```
1 X_train = df.drop(columns=['filename', 'label'])
2 y_train = df['label']
3
4 X_test = df_test.drop(columns=['filename', 'label'])
5 y_test = df_test['label']
```

```
1 #scaling as the ranges of features varies across all
2 #We scale because statistical classifiers (like logistic regression, SVM, KNN, LDA) are sensitive to the scale
3 from sklearn.preprocessing import StandardScaler
4
5 scaler = StandardScaler()
6 X_train_scaled = scaler.fit_transform(X_train)
7 X_test_scaled = scaler.transform(X_test) # Important: use transform, not fit_transform
```

```
1 X_train
```

```
1 X_train_scaled
```

- Models Selection

```
1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.svm import SVC
3 from sklearn.linear_model import LogisticRegression
4
5
6 from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix
7
8 from sklearn.model_selection import GridSearchCV
9
10 from sklearn.metrics import accuracy_score
```

- ✓ Comparing 3 Statistical classifiers

```
1 rf = RandomForestClassifier(n_estimators=100, random_state=42)
2 rf.fit(X_train_scaled, y_train)
3 print("RF:", rf.score(X_test_scaled, y_test))
4
5
6 svm = SVC(kernel='rbf', C=1, gamma='scale')
7 svm.fit(X_train_scaled, y_train)
8 print("SVM:", svm.score(X_test_scaled, y_test))
9
10
11 lr = LogisticRegression(max_iter=1000)
12 lr.fit(X_train_scaled, y_train)
```

```

13 print("LR:", lr.score(X_test_scaled, y_test))
14

```

RF: 0.45128676470588236
 SVM: 0.6323529411764706
 LR: 0.6369485294117647

```

1
2
3 print("SVM Report:")
4 print(classification_report(y_test, svm.predict(X_test_scaled)))
5
6 print("LogReg Report:")
7 print(classification_report(y_test, lr.predict(X_test_scaled)))
8

```

SVM Report:

	precision	recall	f1-score	support
0	0.59	0.87	0.70	544
1	0.76	0.39	0.52	544
accuracy			0.63	1088
macro avg	0.67	0.63	0.61	1088
weighted avg	0.67	0.63	0.61	1088

LogReg Report:

	precision	recall	f1-score	support
0	0.62	0.70	0.66	544
1	0.66	0.57	0.61	544
accuracy			0.64	1088
macro avg	0.64	0.64	0.64	1088
weighted avg	0.64	0.64	0.64	1088

SVM - because lr is less stable

✓ SVM

✓ Tuning & COMPARING svm hyperparameters

ReSeaching The good params - cv=3 gave gamma as scale while 5 giving

```

1
2 from sklearn.model_selection import GridSearchCV
3
4 params = {
5     'C': [0.1, 1, 10, 100],
6     'gamma': ['scale', 'auto', 1, 0.1, 0.01, 0.001],
7     'kernel': ['rbf']
8 }
9
10 grid = GridSearchCV(SVC(), param_grid=params, cv=5, scoring='f1', verbose=1)
11 grid.fit(X_train_scaled, y_train)
12 print("Best Params:", grid.best_params_)
13 print("Best F1 Score:", grid.best_score_)
14 print("Best Mean Test Score:", grid.cv_results_['mean_test_score'])
15 print("Best Mean Train Score:", grid.cv_results_['mean_train_score'])

```

SVM

- Best Params: {'C': 10, 'gamma': 0.01, 'kernel': 'rbf'}
- Best F1 Score: 0.9896433774231864

✓ The SVM model on 86 feaTURES- gamma 0.01 & scale

```

1 svm = SVC(kernel='rbf', C=10, gamma=0.01) #The best svm parameters from tuning and comparing
2 svm.fit(X_train_scaled, y_train)
3 print("SVM:", svm.score(X_test_scaled, y_test))

```

✓ svm model with feature reduced- chroma removed (86-24)

Removing chroma features

```
1 #trimming the features
2 df_reduced = df.loc[:, ~df.columns.str.startswith('chroma')]
3 df_test_reduced = df_test.loc[:, ~df_test.columns.str.startswith('chroma')]
4
5 print(df_reduced.columns.tolist())
6 print(df_reduced.shape)
7
8 print(df_test_reduced.columns.tolist())
9 print(df_test_reduced.shape)
```

```
['filename', 'mfcc0_mean', 'mfcc0_std', 'mfcc1_mean', 'mfcc1_std', 'mfcc2_mean', 'mfcc2_std', 'mfcc3_mean', 'mfcc3_std', 'mfcc4_mean', 'mfcc4_std', 'mfcc5_mean', 'mfcc5_std', 'mfcc6_mean', 'mfcc6_std', 'mfcc7_mean', 'mfcc7_std', 'mfcc8_mean', 'mfcc8_std', 'mfcc9_mean', 'mfcc9_std', 'mfcc10_mean', 'mfcc10_std', 'mfcc11_mean', 'mfcc11_std', 'mfcc12_mean', 'mfcc12_std', 'mfcc13_mean', 'mfcc13_std', 'mfcc14_mean', 'mfcc14_std', 'mfcc15_mean', 'mfcc15_std', 'mfcc16_mean', 'mfcc16_std', 'mfcc17_mean', 'mfcc17_std', 'mfcc18_mean', 'mfcc18_std', 'mfcc19_mean', 'mfcc19_std', 'mfcc20_mean', 'mfcc20_std', 'mfcc21_mean', 'mfcc21_std', 'mfcc22_mean', 'mfcc22_std', 'mfcc23_mean', 'mfcc23_std', 'mfcc24_mean', 'mfcc24_std', 'mfcc25_mean', 'mfcc25_std', 'mfcc26_mean', 'mfcc26_std', 'mfcc27_mean', 'mfcc27_std', 'mfcc28_mean', 'mfcc28_std', 'mfcc29_mean', 'mfcc29_std', 'mfcc30_mean', 'mfcc30_std', 'mfcc31_mean', 'mfcc31_std', 'mfcc32_mean', 'mfcc32_std', 'mfcc33_mean', 'mfcc33_std', 'mfcc34_mean', 'mfcc34_std', 'mfcc35_mean', 'mfcc35_std', 'mfcc36_mean', 'mfcc36_std', 'mfcc37_mean', 'mfcc37_std', 'mfcc38_mean', 'mfcc38_std', 'mfcc39_mean', 'mfcc39_std', 'mfcc40_mean', 'mfcc40_std', 'mfcc41_mean', 'mfcc41_std', 'mfcc42_mean', 'mfcc42_std', 'mfcc43_mean', 'mfcc43_std', 'mfcc44_mean', 'mfcc44_std', 'mfcc45_mean', 'mfcc45_std', 'mfcc46_mean', 'mfcc46_std', 'mfcc47_mean', 'mfcc47_std', 'mfcc48_mean', 'mfcc48_std', 'mfcc49_mean', 'mfcc49_std', 'mfcc50_mean', 'mfcc50_std', 'mfcc51_mean', 'mfcc51_std', 'mfcc52_mean', 'mfcc52_std', 'mfcc53_mean', 'mfcc53_std', 'mfcc54_mean', 'mfcc54_std', 'mfcc55_mean', 'mfcc55_std', 'mfcc56_mean', 'mfcc56_std', 'mfcc57_mean', 'mfcc57_std', 'mfcc58_mean', 'mfcc58_std', 'mfcc59_mean', 'mfcc59_std', 'mfcc60_mean', 'mfcc60_std', 'mfcc61_mean', 'mfcc61_std', 'mfcc62_mean', 'mfcc62_std', 'mfcc63_mean', 'mfcc63_std', 'mfcc64_mean', 'mfcc64_std', 'mfcc65_mean', 'mfcc65_std', 'mfcc66_mean', 'mfcc66_std', 'mfcc67_mean', 'mfcc67_std', 'mfcc68_mean', 'mfcc68_std', 'mfcc69_mean', 'mfcc69_std', 'mfcc70_mean', 'mfcc70_std', 'mfcc71_mean', 'mfcc71_std', 'mfcc72_mean', 'mfcc72_std', 'mfcc73_mean', 'mfcc73_std', 'mfcc74_mean', 'mfcc74_std', 'mfcc75_mean', 'mfcc75_std', 'mfcc76_mean', 'mfcc76_std', 'mfcc77_mean', 'mfcc77_std', 'mfcc78_mean', 'mfcc78_std', 'mfcc79_mean', 'mfcc79_std', 'mfcc80_mean', 'mfcc80_std', 'mfcc81_mean', 'mfcc81_std', 'mfcc82_mean', 'mfcc82_std', 'mfcc83_mean', 'mfcc83_std', 'mfcc84_mean', 'mfcc84_std', 'mfcc85_mean', 'mfcc85_std', 'mfcc86_mean', 'mfcc86_std', 'mfcc87_mean', 'mfcc87_std', 'mfcc88_mean', 'mfcc88_std', 'mfcc89_mean', 'mfcc89_std', 'mfcc90_mean', 'mfcc90_std', 'mfcc91_mean', 'mfcc91_std', 'mfcc92_mean', 'mfcc92_std', 'mfcc93_mean', 'mfcc93_std', 'mfcc94_mean', 'mfcc94_std', 'mfcc95_mean', 'mfcc95_std', 'mfcc96_mean', 'mfcc96_std', 'mfcc97_mean', 'mfcc97_std', 'mfcc98_mean', 'mfcc98_std', 'mfcc99_mean', 'mfcc99_std']
```

Preprocessing new reduced dataset

```
1 X_reduced_train = df_reduced.drop(columns=['filename', 'label'])
2
3 X_reduced_test = df_test_reduced.drop(columns=['filename', 'label'])
```

```
1 X_reduced_train.shape
```

```
1 from sklearn.preprocessing import StandardScaler
2
3 scaler2 = StandardScaler()
4 X_reduced_train_scaled = scaler2.fit_transform(X_reduced_train)
5 X_reduced_test_scaled = scaler2.transform(X_reduced_test) # Important: use transform, not fit_transform
```

```
1 X_reduced_train
```

1 X reduced_train_scaled

Training and comparing

```
1 svm_reduced = SVC(kernel='rbf', C=10, gamma=0.01)
2 svm_reduced.fit(X_reduced_train_scaled, y_train)
3 print("SVM:", svm_reduced.score(X_reduced_test_scaled, y_test))
```

SVM: 0.5854779411764706

```
1 lr_reduced = LogisticRegression(C=0.1, max_iter=500, penalty='l1', solver='liblinear')
2 lr_reduced.fit(X_reduced_train_scaled, y_train)
3 print("LR:", lr_reduced.score(X_reduced_test_scaled, y_test))
```

LR: 0.5900735294117647

CONCLUSION : NOT TO REDUCE FEATURE SET as it cost ACCURACY ~ Thus not saving the reduceSVM model

svm model with feature reduced- PCA applied ()

▼ LR

- ✓ Tuning and choosing hyperparameters

```
1 params = {
2     'C': [0.001, 0.01, 0.1, 1, 10, 100], # Regularization strength (inverse of lambda)
3     'penalty': ['l1', 'l2', 'elasticnet'], # Different regularization types
4     'solver': ['liblinear', 'saga'], # solvers that support l1 and elasticnet
5     'max_iter': [500, 1000]
6 }
7
8
9 log_reg = LogisticRegression()
10 grid = GridSearchCV(log_reg, param_grid=params, cv=5, scoring='f1', verbose=1, n_jobs=-1)
11 grid.fit(X_train_scaled, y_train)
12
13 # Display best parameters and best score
```

```

14 print("Best Parameters:", grid.best_params_)
15 print("Best F1 Score:", grid.best_score_)
16
17 # Best model
18 best_lr = grid.best_estimator_
19

```

LR

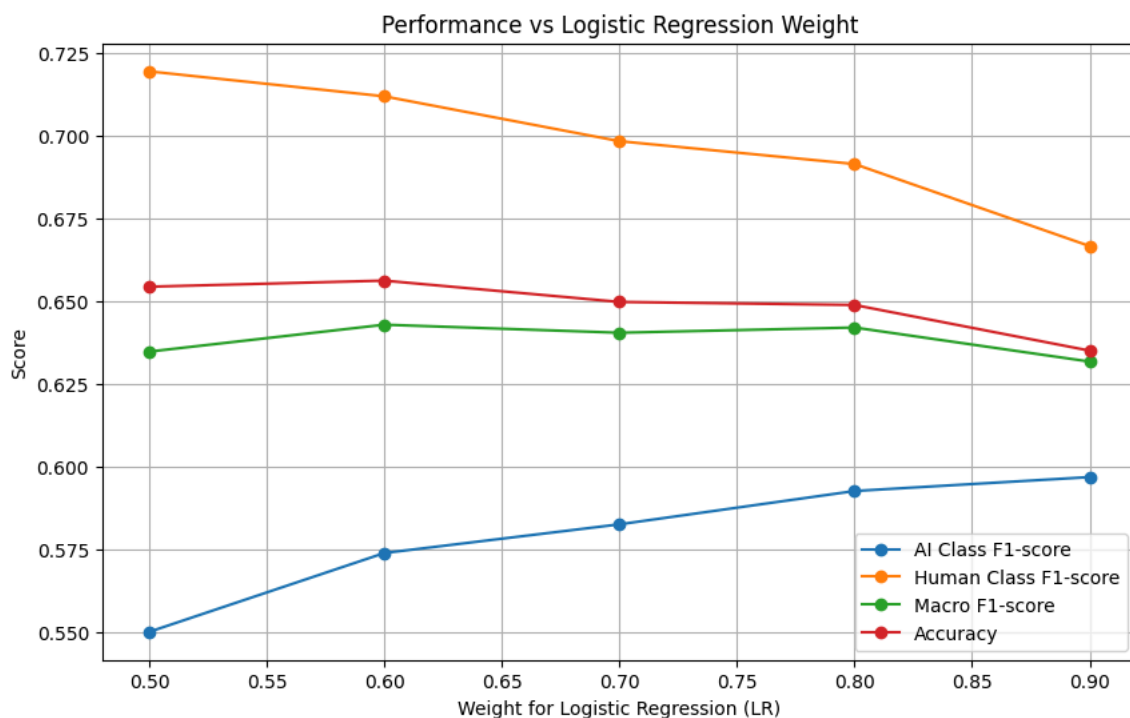
- Best Parameters: {'C': 0.1, 'max_iter': 500, 'penalty': 'l1', 'solver': 'liblinear'}
- Best F1 Score: 0.9098175641027335

✓ LORIS - ensembled LR & SVM

```

1
2 # -----
3 # Plot Performance vs LR Weight
4 # -----
5 plt.figure(figsize=(10,6))
6 plt.plot(weights, ai_f1_scores, marker='o', label='AI Class F1-score')
7 plt.plot(weights, human_f1_scores, marker='o', label='Human Class F1-score')
8 plt.plot(weights, macro_f1_scores, marker='o', label='Macro F1-score')
9 plt.plot(weights, accuracies, marker='o', label='Accuracy')
10
11 plt.title("Performance vs Logistic Regression Weight")
12 plt.xlabel("Weight for Logistic Regression (LR)")
13 plt.ylabel("Score")
14 plt.grid(True)
15 plt.legend()
16 plt.show()

```



✓ MODEL

```

1 model_path = '/content/drive/MyDrive/AiAudio_PR_Project/AiAudio_Model'
2 svm_m_path = model_path+'/svm' #will use it to train and select best model
3 svm_reduced_m_path = model_path+'/svm_reduced'
4 lr_m_path = model_path+'/lr'
5 lr_reduced_m_path = model_path + '/lr_reduced' #retrain, for our scenario, on the selected best model
6 loris1_m_path = model_path + '/loris1'
7 loris2_m_path = model_path + '/loris2'
8
9 print("Path to SVM Model:", svm_m_path)
10 print("Path to SVM Reduced Model:", svm_reduced_m_path)
11 print("Path to LR Model:", lr_m_path)

```

Path to SVM Model: /content/drive/MyDrive/AiAudio_PR_Project/AiAudio_Model/svm

Path to SVM Reduced Model: /content/drive/MyDrive/AiAudio_PR_Project/AiAudio_Model/svm_reduced

▼ Training - no need run for loaded models

```
1 svm = SVC(kernel='rbf', C=10, gamma=0.01) #The best svm parameters from tuning and comparing
2 svm.fit(X_train_scaled, y_train)
3 svm_y_pred = svm.predict(X_test_scaled)
4 svm_acc = accuracy_score(y_test, svm_y_pred)
5 svm_f1 = f1_score(y_test, svm_y_pred, average='weighted')
6
7 print("SVM ACCURACY:", svm_acc)
8 print("SVM f1 score:", svm_f1)
9 print("SVM:", svm.score(X_test_scaled, y_test))
```

SVM ACCURACY: 0.6332720588235294
SVM f1 score: 0.6001823582233887
SVM: 0.6332720588235294

```
1 X_test_scaled.shape
```

```
1 svm_reduced = SVC(kernel='rbf', C=10, gamma=0.01)
2 svm_reduced.fit(X_reduced_train_scaled, y_train)
3 svm_reduced_y_pred = svm_reduced.predict(X_reduced_test_scaled)
4 svm_reduced_acc = accuracy_score(y_test, svm_reduced_y_pred)
5 svm_reduced_f1 = f1_score(y_test, svm_reduced_y_pred, average='weighted')
6
7 print("SVM ACCURACY:", svm_reduced_acc)
8 print("SVM f1 score:", svm_reduced_f1)
9 print("SVM:", svm_reduced.score(X_reduced_test_scaled, y_test))
```

SVM ACCURACY: 0.5854779411764706
SVM f1 score: 0.538538654328128
SVM: 0.5854779411764706

```
1 lr = LogisticRegression(C=0.1, max_iter=500, penalty='l1', solver='liblinear')
2 lr.fit(X_train_scaled, y_train)
3 lr_y_pred = lr.predict(X_test_scaled)
4 lr_acc = accuracy_score(y_test, lr_y_pred)
5 lr_f1 = f1_score(y_test, lr_y_pred, average='weighted')
6
7 print("LR ACCURACY:", lr_acc)
8 print("LR f1 score:", lr_f1)
```

LR ACCURACY: 0.6378676470588235
LR f1 score: 0.6357988819221423

```
1 lr_reduced = LogisticRegression(C=0.1, max_iter=500, penalty='l1', solver='liblinear')
2 lr_reduced.fit(X_reduced_train_scaled, y_train)
3 lr_reduced_y_pred = lr_reduced.predict(X_reduced_test_scaled)
4 lr_reduced_acc = accuracy_score(y_test, lr_reduced_y_pred)
5 lr_reduced_f1 = f1_score(y_test, lr_reduced_y_pred, average='weighted')
6
7 print("LR reduced ACCURACY:", lr_reduced_acc)
8 print("le reduced f1 score:", lr_reduced_f1)
9 print("LR REDUCED:", lr_reduced.score(X_reduced_test_scaled, y_test))
```

LR reduced ACCURACY: 0.5900735294117647
le reduced f1 score: 0.589934963966522
LR REDUCED: 0.5900735294117647

▼ Saving Model

SVM

```
1 import pickle
2
3 with open(svm_m_path, "wb") as f:
4     pickle.dump(svm, f)
5
6 print(f"Model saved at: {svm_m_path}")
7
```

Model saved at: /content/drive/MyDrive/AiAudio_PR_Project/AiAudio_Model/svm

LR

```

1 import pickle
2
3 with open(lr_m_path, "wb") as f:
4     pickle.dump(lr, f)
5
6 print(f"Model saved at: {lr_m_path}")
7

```

Model saved at: /content/drive/MyDrive/AiAudio_PR_Project/AiAudio_Model/lr

SVM_REDUCED (NO CHROMA)

```

1 import pickle
2
3 with open(svm_reduced_m_path, "wb") as f:
4     pickle.dump(svm_reduced, f)
5
6 print(f"Model saved at: {svm_reduced_m_path}")
7

```

Model saved at: /content/drive/MyDrive/AiAudio_PR_Project/AiAudio_Model/svm_reduced

LR_Reduced

```

1 import pickle
2
3 with open(lr_reduced_m_path, "wb") as f:
4     pickle.dump(lr_reduced, f)
5
6 print(f"Model saved at: {lr_reduced_m_path}")
7

```

Model saved at: /content/drive/MyDrive/AiAudio_PR_Project/AiAudio_Model/lr_reduced

Loris 1 and 2

```

1 import pickle
2
3 with open(loris1_m_path, "wb") as f:
4     pickle.dump(loris1, f)
5
6 print(f"Model saved at: {loris1_m_path}")

```

Model saved at: /content/drive/MyDrive/AiAudio_PR_Project/AiAudio_Model/loris1

```

1 import pickle
2
3 with open(loris2_m_path, "wb") as f:
4     pickle.dump(loris2, f)
5
6 print(f"Model saved at: {loris2_m_path}")

```

Model saved at: /content/drive/MyDrive/AiAudio_PR_Project/AiAudio_Model/loris2

▼ Loading Model

```

1 import pickle
2
3 with open(svm_m_path, "rb") as f:
4     svm = pickle.load(f)

```

```

1 import pickle
2
3 with open(svm_reduced_m_path, "rb") as f:
4     svm_reduced = pickle.load(f)

```

```

1 import pickle
2
3 with open(lr_reduced_m_path, "rb") as f:
4     lr_reduced = pickle.load(f)

```

```

1 import pickle
2

```

```
3 with open(loris1_m_path, "rb") as f:
4     loris1 = pickle.load(f)
```

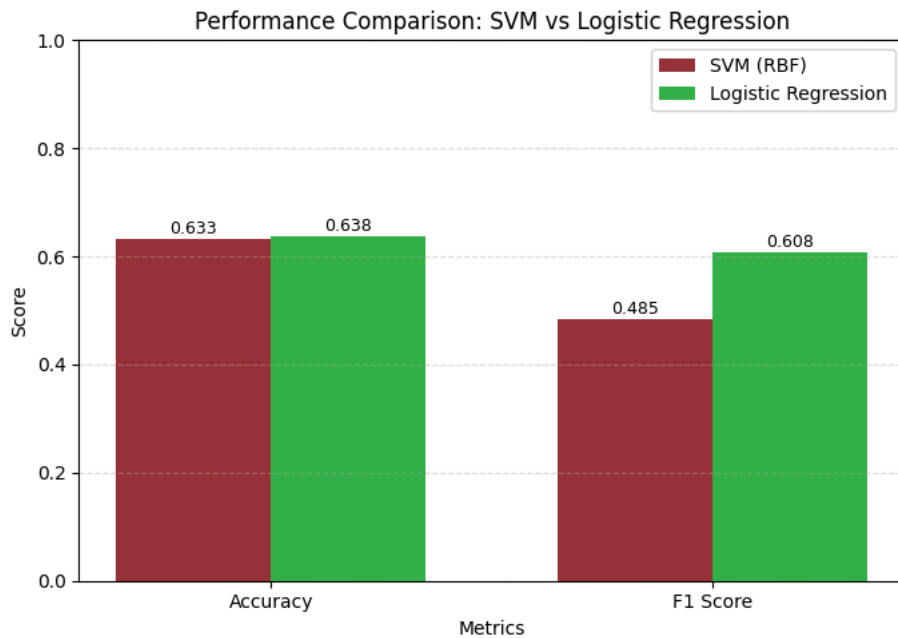
```
1 import pickle
2
3 with open(loris1_m_path, "rb") as f:
4     loris1 = pickle.load(f)
```

```
1 import pickle
2
3 with open(loris2_m_path, "rb") as f:
4     loris2 = pickle.load(f)
```

✓ Comparision PLOTS

✓ LR VS SVM Plots

```
1 import matplotlib.pyplot as plt
2 from sklearn.metrics import accuracy_score, f1_score
3
4 # Compute metrics
5 svm_acc = accuracy_score(y_test, svm_y_pred)
6 svm_f1 = f1_score(y_test, svm_y_pred)
7 lr_acc = accuracy_score(y_test, lr_y_pred)
8 lr_f1 = f1_score(y_test, lr_y_pred)
9
10 # Prepare data
11 metrics = ['Accuracy', 'F1 Score']
12 svm_scores = [svm_acc, svm_f1]
13 lr_scores = [lr_acc, lr_f1]
14
15 # --- Plot ---
16 plt.figure(figsize=(7,5))
17 bar_width = 0.35
18 x = range(len(metrics))
19
20 # Custom colors
21 svm_color = '#7d000a'
22 lr_color = '#009c1a' # orange
23
24 # Bars
25 bars1 = plt.bar(x, svm_scores, width=bar_width, label='SVM (RBF)', color=svm_color, alpha=0.8)
26 bars2 = plt.bar([i + bar_width for i in x], lr_scores, width=bar_width, label='Logistic Regression', color=lr_color, alpha=0.8)
27
28 # Labels and formatting
29 plt.xlabel('Metrics')
30 plt.ylabel('Score')
31 plt.title('Performance Comparison: SVM vs Logistic Regression')
32 plt.xticks([i + bar_width/2 for i in x], metrics)
33 plt.ylim(0, 1) # zoomed-in range
34 plt.legend()
35 plt.grid(axis='y', linestyle='--', alpha=0.4)
36
37 # Add values on top of bars
38 for bars in [bars1, bars2]:
39     for bar in bars:
40         yval = bar.get_height()
41         plt.text(bar.get_x() + bar.get_width()/2, yval + 0.002, f"{yval:.3f}", ha='center', va='bottom', fontweight='bold')
42
43 plt.tight_layout()
44 plot_path = "/content/drive/MyDrive/AiAudio_PR_Project/AiAudio_Plot"
45 plt.savefig(plot_path+"svm_vs_lr.png", dpi=300, bbox_inches='tight')
46 plt.show()
47
```

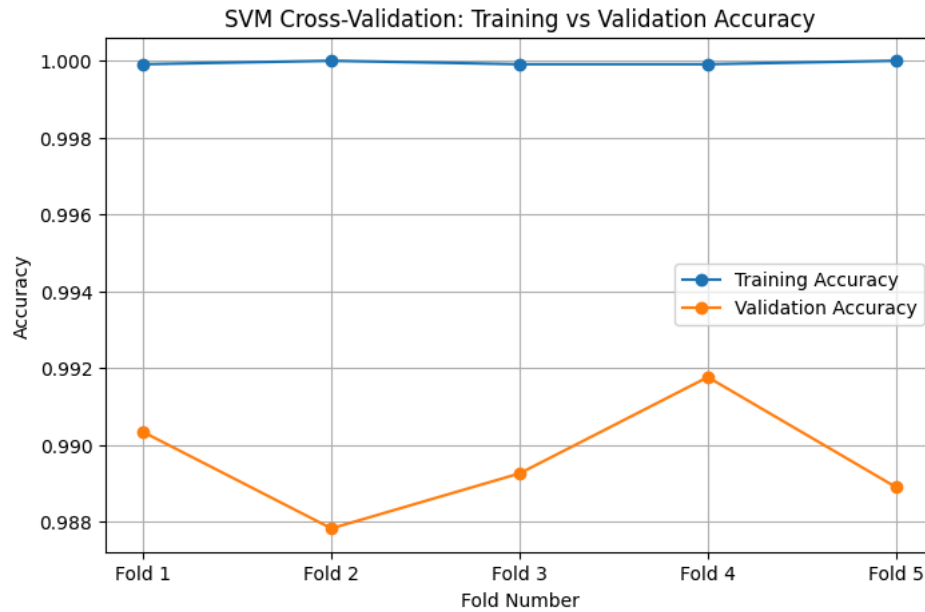


```
1 import numpy as np
2 from sklearn.model_selection import cross_validate
3 from sklearn.svm import SVC
4 import matplotlib.pyplot as plt
```

```
1 import numpy as np
2 from sklearn.model_selection import cross_validate
3 from sklearn.svm import SVC
4 import matplotlib.pyplot as plt
5
6
7 cv_results = cross_validate(
8     svm,
9     X_train_scaled,
10    y_train,
11    cv=5,
12    return_train_score=True
13 )
```

```
1 svm_train_acc = cv_results['train_score']
2 svm_val_acc = cv_results['test_score']
3
4 print("Training accuracy per fold:", svm_train_acc)
5 print("Validation accuracy per fold:", svm_val_acc)
6 print("Mean Training Accuracy:", np.mean(svm_train_acc))
7 print("Mean Validation Accuracy:", np.mean(svm_val_acc))
8
9
10 plt.figure(figsize=(8,5))
11 plt.plot(svm_train_acc, marker='o', label='Training Accuracy')
12 plt.plot(svm_val_acc, marker='o', label='Validation Accuracy')
13 plt.title("SVM Cross-Validation: Training vs Validation Accuracy")
14 plt.xlabel("Fold Number")
15 plt.ylabel("Accuracy")
16 plt.xticks(range(5), ['Fold 1', 'Fold 2', 'Fold 3', 'Fold 4', 'Fold 5'])
17 plt.legend()
18 plt.grid(True)
19 plt.show()
20
```

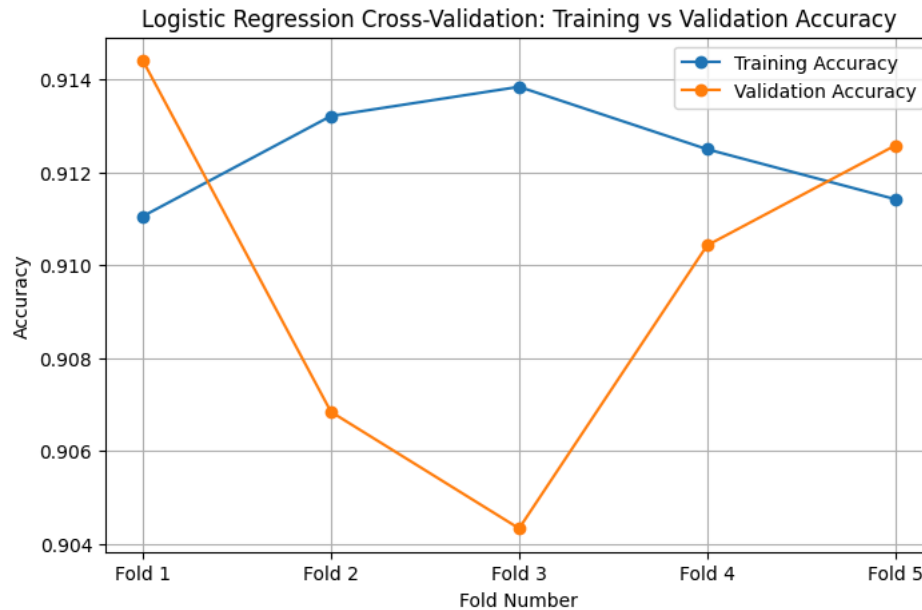
Training accuracy per fold: [0.99991043 1. 0.99991043 0.99991043 1.]
Validation accuracy per fold: [0.99032951 0.98781799 0.98925116 0.99175923 0.98889287]
Mean Training Accuracy: 0.9999462590313726
Mean Validation Accuracy: 0.9896101519517811



```
1 import numpy as np
2 from sklearn.model_selection import cross_validate
3 from sklearn.svm import SVC
4 import matplotlib.pyplot as plt
5
6
7 cv_results = cross_validate(
8     lr,
9     X_train_scaled,
10    y_train,
11    cv=5,
12    return_train_score=True
13 )
```

```
1 lr_train_acc = cv_results['train_score']
2 lr_val_acc = cv_results['test_score']
3
4 print("Training accuracy per fold:", lr_train_acc)
5 print("Validation accuracy per fold:", lr_val_acc)
6 print("Mean Training Accuracy:", np.mean(lr_train_acc))
7 print("Mean Validation Accuracy:", np.mean(lr_val_acc))
8
9
10 plt.figure(figsize=(8,5))
11 plt.plot(lr_train_acc, marker='o', label='Training Accuracy')
12 plt.plot(lr_val_acc, marker='o', label='Validation Accuracy')
13 plt.title("Logistic Regression Cross-Validation: Training vs Validation Accuracy")
14 plt.xlabel("Fold Number")
15 plt.ylabel("Accuracy")
16 plt.xticks(range(5), ['Fold 1', 'Fold 2', 'Fold 3', 'Fold 4', 'Fold 5'])
17 plt.legend()
18 plt.grid(True)
19 plt.show()
20
```

Training accuracy per fold: [0.91105339 0.91321093 0.91383789 0.9124944 0.91141961]
 Validation accuracy per fold: [0.91439828 0.90684343 0.90433536 0.91042637 0.91257614]
 Mean Training Accuracy: 0.9124032432312742
 Mean Validation Accuracy: 0.9097159155656896



✓ Prediction of Audio

```
1 #IMPORTANT
2 import warnings
3 warnings.filterwarnings("ignore", message="X does not have valid feature names")
4 #we get the warning, difficult to see output, the warning is not error, does not affect the performance or a
```

```
1 #Sample Path
2 import os
3 sample_path = "/content/drive/MyDrive/AiAudio_PR_Project/Self_Generated_Test_Samples"
4 audio_files = [f for f in os.listdir(sample_path) if f.endswith(('.mp3', '.webm'))]
```

```
1 audio_files
```

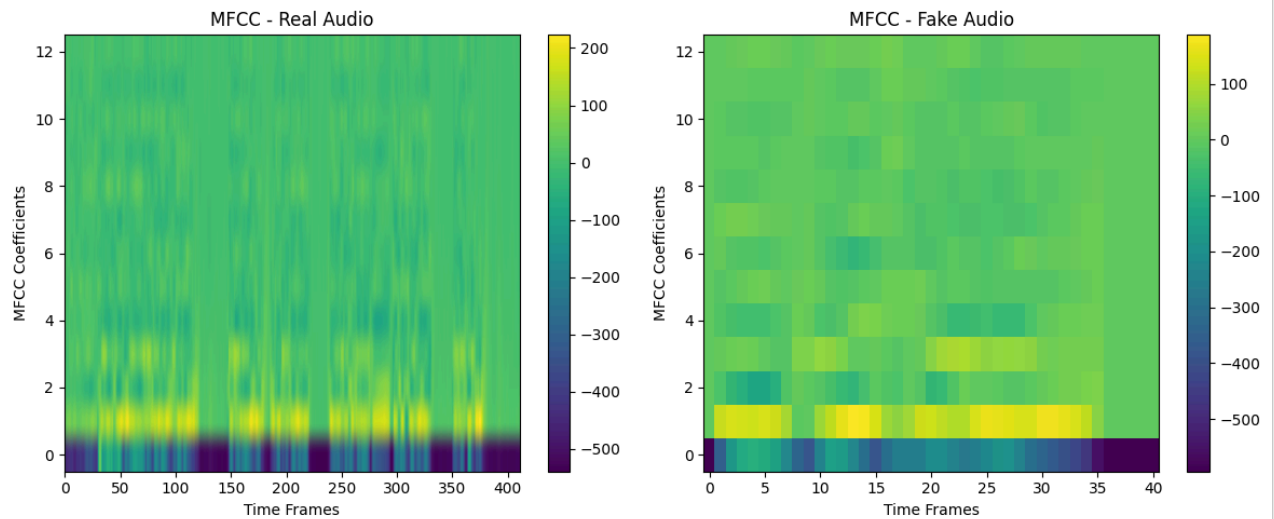
```
['ROOM IITI 1.mp3',
 'Room IITI 2.mp3',
 'voice over for navtri night.mp3',
 '(AI) After cleaning.mp3',
 '(AI) Bapunagar nahee, babu nadee yes.mp3',
 '(AI) Before cleaning.mp3']
```

```
1 #MFCC of real and fake
2 import librosa
3 import librosa.display
4 import matplotlib.pyplot as plt
5 import numpy as np
6 import os
7
8 def plot_mfcc(audio_path, title):
9     y, sr = librosa.load(audio_path, sr=16000)
10    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
11
12    plt.imshow(mfcc, aspect='auto', origin='lower')
13    plt.colorbar()
14    plt.title(title)
15    plt.xlabel("Time Frames")
16    plt.ylabel("MFCC Coefficients")
17
18 # ENTER YOUR FILE PATHS HERE
19 real_audio_path = os.path.join(sample_path, 'ROOM IITI 1.mp3')
20 fake_audio_path = os.path.join(sample_path, '(AI) After cleaning.mp3')
21
22 plt.figure(figsize=(12, 5))
23
24 plt.subplot(1, 2, 1)
25 plot_mfcc(real_audio_path, "MFCC - Real Audio")
26
27 plt.subplot(1, 2, 2)
```

```

28 plot_mfcc(fake_audio_path, "MFCC - Fake Audio")
29
30 plt.tight_layout()
31 plt.show()
32

```



OWN SAMPLES

REAL IS 0 and FAKE IS 1 - Testing on mixed samples (3 real , 3 fake)

```

1 def single_audio_features(file_path):
2     y, sr = librosa.load(file_path, sr=16000)
3
4
5     mfccs = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
6
7     delta_mfccs = librosa.feature.delta(mfccs)
8
9     spec_cent = librosa.feature.spectral_centroid(y=y, sr=sr)
10
11     spec_bw = librosa.feature.spectral_bandwidth(y=y, sr=sr)
12
13     chroma = librosa.feature.chroma_stft(y=y, sr=sr)
14
15     zcr = librosa.feature.zero_crossing_rate(y)
16
17     rms = librosa.feature.rms(y=y)
18
19
20     features = []
21     for feature_set in [mfccs, delta_mfccs, spec_cent, spec_bw, chroma, zcr, rms]:
22         features.extend(np.mean(feature_set, axis=1))
23         features.extend(np.std(feature_set, axis=1))
24
25     return features

```

```

1 def reduced_single_audio_features(file_path):
2     y, sr = librosa.load(file_path, sr=16000)
3
4
5     mfccs = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
6
7     delta_mfccs = librosa.feature.delta(mfccs)
8
9     spec_cent = librosa.feature.spectral_centroid(y=y, sr=sr)
10
11     spec_bw = librosa.feature.spectral_bandwidth(y=y, sr=sr)
12
13     # chroma = librosa.feature.chroma_stft(y=y, sr=sr)
14
15     zcr = librosa.feature.zero_crossing_rate(y)
16

```

```

17     rms = librosa.feature.rms(y=y)
18
19
20     features = []
21     for feature_set in [mfccs, delta_mfccs, spec_cent, spec_bw, zcr, rms]:
22         features.extend(np.mean(feature_set, axis=1))
23         features.extend(np.std(feature_set, axis=1))
24
25     return features

```

```

1 import warnings
2 warnings.filterwarnings("ignore", message="X does not have valid feature names")
3
4 for file in audio_files:
5     file_path = os.path.join(sample_path, file)
6     features = single_audio_features(file_path)
7     features_scaled = scaler.transform([features])
8
9     prediction = svm.predict(features_scaled)[0]
10    label = "AI-generated" if prediction == 1 else "Human"
11
12    print(f"{file}: {label}")
13

```

```

ROOM IITI 1.mp3: Human
Room IITI 2.mp3: Human
voice over for navtri night.mp3: Human
(AI) After cleaning.mp3: Human
(AI) Bapunagar nahee, bapu nadee yes.mp3: AI-generated
(AI) Before cleaning.mp3: Human

```

SVM 4/6 accuracy

```

1 import warnings
2 warnings.filterwarnings("ignore", message="X does not have valid feature names")
3
4 for file in audio_files:
5     file_path = os.path.join(sample_path, file)
6     features = single_audio_features(file_path)
7     features_scaled = scaler.transform([features])
8
9     prediction = lr.predict(features_scaled)[0]
10    label = "AI-generated" if prediction == 1 else "Human"
11
12    print(f"{file}: {label}")
13

```

```

ROOM IITI 1.mp3: Human
Room IITI 2.mp3: Human
voice over for navtri night.mp3: Human
(AI) After cleaning.mp3: AI-generated
(AI) Bapunagar nahee, bapu nadee yes.mp3: AI-generated
(AI) Before cleaning.mp3: AI-generated

```

LR 100% accuracy

Add blockquote

▼ FOR-RERECORDED VALIDATION SAMPLES

```

1 #DOWNLOAD For dataset from kaggle first
2 print("Path to rerec real files:", for_rerec_path_real)
3 print("Path to rerec fake files:", for_rerec_path_fake)

```

```

Path to rerec real files: /root/.cache/kagglehub/datasets/mohammedabdeldayem/the-fake-or-real-dataset/versions/2/
Path to rerec fake files: /root/.cache/kagglehub/datasets/mohammedabdeldayem/the-fake-or-real-dataset/versions/2/

```

- REAL 1101 files (wav)

```

1 #SVM
2 correct = 0
3 total = 0
4
5 for file_name in os.listdir(for_rerec_path_real):
6     if file_name.endswith(".wav"):
7         file_path = os.path.join(for_rerec_path_real, file_name)

```

```

8
9     features = single_audio_features(file_path)
10
11     features_scaled = scaler.transform([features])
12
13     pred = svm.predict(features_scaled)[0]
14
15     if pred == 0:
16         correct += 1
17     total += 1
18
19 svm_real_accuracy = correct / total if total > 0 else 0
20 print(f"Predicted {correct}/{total} correctly as real (accuracy = {svm_real_accuracy*100:.2f}%)")
21

```

Predicted 980/1101 correctly as real (accuracy = 89.01%)

```

1 #SVM reduced
2 correct = 0
3 total = 0
4
5
6 for file_name in os.listdir(for_rerec_path_real):
7     if file_name.endswith(".wav"):
8         file_path = os.path.join(for_rerec_path_real, file_name)
9
10        features = reduced_single_audio_features(file_path)
11
12        features_scaled = scaler2.transform([features])
13
14        pred = svm_reduced.predict(features_scaled)[0]
15
16        if pred == 0:
17            correct += 1
18        total += 1
19
20 svm_reduced_real_accuracy = correct / total if total > 0 else 0
21 print(f"Predicted {correct}/{total} correctly as real (accuracy = {svm_reduced_real_accuracy*100:.2f}%)")
22

```

Predicted 999/1101 correctly as real (accuracy = 90.74%)

```

1 #LR
2 correct = 0
3 total = 0
4
5 for file_name in os.listdir(for_rerec_path_real):
6     if file_name.endswith(".wav"):
7         file_path = os.path.join(for_rerec_path_real, file_name)
8
9         features = single_audio_features(file_path)
10
11         features_scaled = scaler.transform([features])
12
13         pred = lr.predict(features_scaled)[0]
14
15         if pred == 0:
16             correct += 1
17         total += 1
18
19 lr_real_accuracy = correct / total if total > 0 else 0
20 print(f"Predicted {correct}/{total} correctly as real (accuracy = {lr_real_accuracy*100:.2f}%)")
21

```

Predicted 922/1101 correctly as real (accuracy = 83.74%)

```

1 #LR_REDUCED
2 correct = 0
3 total = 0
4
5
6 for file_name in os.listdir(for_rerec_path_real):
7     if file_name.endswith(".wav"):
8         file_path = os.path.join(for_rerec_path_real, file_name)
9
10        features = reduced_single_audio_features(file_path)
11
12        features_scaled = scaler2.transform([features])
13
14        pred = lr_reduced.predict(features_scaled)[0]
15

```

```

16         if pred == 0:
17             correct += 1
18             total += 1
19
20 lr_reduced_real_accuracy = correct / total if total > 0 else 0
21 print(f"Predicted {correct}/{total} correctly as real (accuracy = {lr_reduced_real_accuracy*100:.2f}%)")
22

```

Predicted 765/1101 correctly as real (accuracy = 69.48%)

```

1 #LORIS -0.3 svm, 0.7 lr
2 import warnings
3 warnings.filterwarnings("ignore", message="X does not have valid feature names")
4 correct = 0
5 total = 0
6
7 for file_name in os.listdir(for_rerec_path_real):
8     if file_name.endswith(".wav"):
9         file_path = os.path.join(for_rerec_path_real, file_name)
10
11         features = single_audio_features(file_path)
12         features_scaled = scaler.transform([features])
13
14         pred = loris1.predict(features_scaled)[0]
15
16         if pred == 0:
17             correct += 1
18             total += 1
19
20
21 loris1_real_accuracy = correct / total if total > 0 else 0
22 print(f"Predicted {correct}/{total} correctly as real (accuracy = {loris1_real_accuracy*100:.2f}%)")

```

Predicted 978/1101 correctly as real (accuracy = 88.83%)

```

1 #LORIS2 - 0.1 svm, 0.9 lr
2
3 correct = 0
4 total = 0
5
6 for file_name in os.listdir(for_rerec_path_real):
7     if file_name.endswith(".wav"):
8         file_path = os.path.join(for_rerec_path_real, file_name)
9
10        features = single_audio_features(file_path)
11        features_scaled = scaler.transform([features])
12
13        pred = loris2.predict(features_scaled)[0]
14
15        if pred == 0:
16            correct += 1
17            total += 1
18
19
20 loris2_real_accuracy = correct / total if total > 0 else 0
21 print(f"Predicted {correct}/{total} correctly as real (accuracy = {loris2_real_accuracy*100:.2f}%)")

```

Predicted 932/1101 correctly as real (accuracy = 84.65%)

- fake 1143 files (wav)

```

1 #SVM
2 correct = 0
3 total = 0
4
5 for file_name in os.listdir(for_rerec_path_fake):
6     if file_name.endswith(".wav"):
7         file_path = os.path.join(for_rerec_path_fake, file_name)
8
9         features = single_audio_features(file_path)
10        features_scaled = scaler.transform([features])
11
12        pred = svm.predict(features_scaled)[0]
13
14        if pred == 1:
15            correct += 1
16            total += 1
17
18
19 svm_fake_accuracy = correct / total if total > 0 else 0

```

```
20 print(f"Predicted {correct}/{total} correctly as fake (accuracy = {svm_fake_accuracy*100:.2f}%)")
21
```

Predicted 382/1143 correctly as fake (accuracy = 33.42%)

```
1 #SVM reduced
2 correct = 0
3 total = 0
4
5
6 for file_name in os.listdir(for_rerec_path_fake):
7     if file_name.endswith(".wav"):
8         file_path = os.path.join(for_rerec_path_fake, file_name)
9
10        features = reduced_single_audio_features(file_path)
11
12        features_scaled = scaler2.transform([features])
13
14        pred = svm_reduced.predict(features_scaled)[0]
15
16        if pred == 1:
17            correct += 1
18            total += 1
19
20 svm_reduced_fake_accuracy = correct / total if total > 0 else 0
21 print(f"Predicted {correct}/{total} correctly as fake (accuracy = {svm_reduced_fake_accuracy*100:.2f}%)")
22
```

Predicted 342/1143 correctly as fake (accuracy = 29.92%)

```
1 #LR
2 correct = 0
3 total = 0
4
5 for file_name in os.listdir(for_rerec_path_fake):
6     if file_name.endswith(".wav"):
7         file_path = os.path.join(for_rerec_path_fake, file_name)
8
9         features = single_audio_features(file_path)
10
11        features_scaled = scaler.transform([features])
12
13        pred = lr.predict(features_scaled)[0]
14
15        if pred == 1:
16            correct += 1
17            total += 1
18
19 lr_fake_accuracy = correct / total if total > 0 else 0
20 print(f"Predicted {correct}/{total} correctly as fake (accuracy = {lr_fake_accuracy*100:.2f}%)")
21
```

Predicted 927/1143 correctly as fake (accuracy = 81.10%)

```
1 #lr reduced
2 correct = 0
3 total = 0
4
5
6 for file_name in os.listdir(for_rerec_path_fake):
7     if file_name.endswith(".wav"):
8         file_path = os.path.join(for_rerec_path_fake, file_name)
9
10        features = reduced_single_audio_features(file_path)
11
12        features_scaled = scaler2.transform([features])
13
14        pred = lr_reduced.predict(features_scaled)[0]
15
16        if pred == 1:
17            correct += 1
18            total += 1
19
20 lr_reduced_fake_accuracy = correct / total if total > 0 else 0
21 print(f"Predicted {correct}/{total} correctly as fake (accuracy = {lr_reduced_fake_accuracy*100:.2f}%)")
22
```

Predicted 920/1143 correctly as fake (accuracy = 80.49%)

```
1 #LORIS
2 correct = 0
```

```

3 total = 0
4
5 for file_name in os.listdir(for_rerec_path_fake):
6     if file_name.endswith(".wav"):
7         file_path = os.path.join(for_rerec_path_fake, file_name)
8
9         features = single_audio_features(file_path)
10
11         features_scaled = scaler.transform([features])
12
13         pred = loris1.predict(features_scaled)[0]
14
15         if pred == 1:
16             correct += 1
17         total += 1
18
19 loris1_fake_accuracy = correct / total if total > 0 else 0
20 print(f"Predicted {correct}/{total} correctly as fake (accuracy = {loris1_fake_accuracy*100:.2f}%)")

```

Predicted 828/1143 correctly as fake (accuracy = 72.44%)

```

1 #LORIS2
2 correct = 0
3 total = 0
4
5 for file_name in os.listdir(for_rerec_path_fake):
6     if file_name.endswith(".wav"):
7         file_path = os.path.join(for_rerec_path_fake, file_name)
8
9         features = single_audio_features(file_path)
10
11         features_scaled = scaler.transform([features])
12
13         pred = loris2.predict(features_scaled)[0]
14
15         if pred == 1:
16             correct += 1
17         total += 1
18
19 loris2_fake_accuracy = correct / total if total > 0 else 0
20 print(f"Predicted {correct}/{total} correctly as fake (accuracy = {loris2_fake_accuracy*100:.2f}%)")

```

Predicted 901/1143 correctly as fake (accuracy = 78.83%)

CONCLUSION:

- SEEMS LIKE **SVM** IS HUMAN BIASED, Achieving more accuracy in REAL audio prediction and bad performance in AI Generated.
- LR gave lesser accuracy for REAL audio prediction than SVM but doesn't degrade when it comes to Fake audio prediction staying more stable.

```

1 print(" ----- REAL ACCURACIES ----- ")
2 print("SVM:", svm_real_accuracy)
3 print("SVM Reduced:", svm_reduced_real_accuracy)
4 print("LR:", lr_real_accuracy)
5 print("LR Reduced:", lr_reduced_real_accuracy)
6
7 print("LORIS1:", loris1_real_accuracy)
8 print("LORIS2:", loris2_real_accuracy)

```

```

----- REAL ACCURACIES -----
SVM: 0.8900999091734787
SVM Reduced: 0.9073569482288828
LR: 0.8374205267938238
LR Reduced: 0.6948228882833788
LORIS1: 0.888283378746594
LORIS2: 0.846503178928247

```

```

1 print(" ----- FAKE ACCURACIES ----- ")
2 print("SVM:", svm_fake_accuracy)
3 print("SVM Reduced:", svm_reduced_fake_accuracy)
4 print("LR:", lr_fake_accuracy)
5 print("LR Reduced:", lr_reduced_fake_accuracy)
6
7 print("LORIS1:", loris1_fake_accuracy)
8 print("LORIS2:", loris2_fake_accuracy)

```

```

----- FAKE ACCURACIES -----
SVM: 0.3342082239720035
SVM Reduced: 0.2992125984251969
LR: 0.8110236220472441
LR Reduced: 0.8048993875765529

```

LORIS1: 0.7244094488188977
LORIS2: 0.7882764654418197

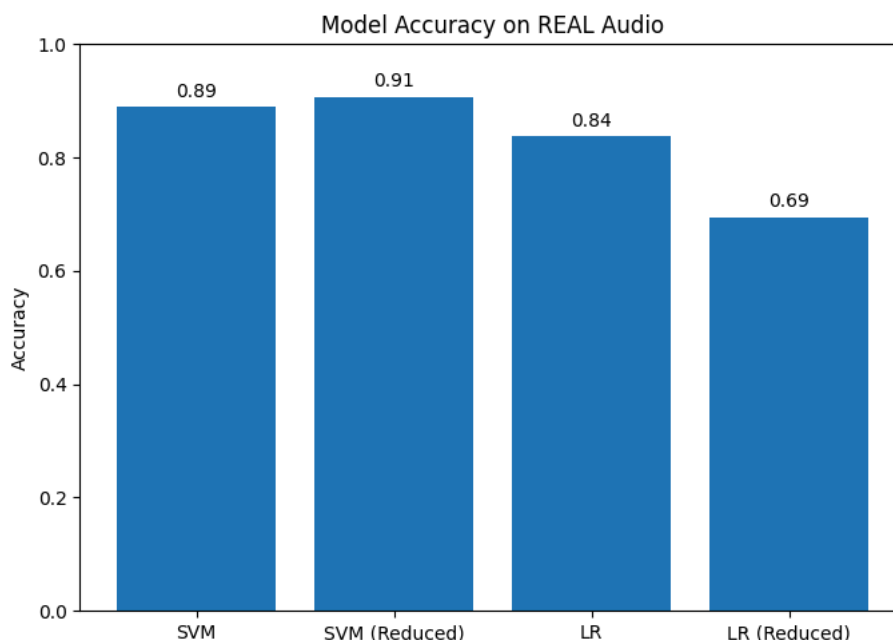
----- REAL ACCURACIES -----

- SVM: 0.8900999091734787
- SVM Reduced: 0.9073569482288828
- LR: 0.8374205267938238
- LR Reduced: 0.6948228882833788
- LORIS1: 0.888283378746594
- LORIS2: 0.846503178928247

----- FAKE ACCURACIES -----

- SVM: 0.3342082239720035
- SVM Reduced: 0.2992125984251969
- LR: 0.8110236220472441
- LR Reduced: 0.8048993875765529
- LORIS1: 0.7244094488188977
- LORIS2: 0.7882764654418197

```
1 # -----  
2 # Plot 1: Accuracy for Real Audio  
3 # -----  
4 plt.figure(figsize=(7, 5))  
5 bars = plt.bar(models, real_acc)  
6 plt.ylabel("Accuracy")  
7 plt.title("Model Accuracy on REAL Audio")  
8 plt.ylim(0, 1)  
9  
10 # Add values on top of bars  
11 for bar in bars:  
12     yval = bar.get_height()  
13     plt.text(bar.get_x() + bar.get_width()/2, yval + 0.01, f"{yval:.2f}",  
14             ha="center", va="bottom")  
15  
16 plt.tight_layout()  
17 plot_path = "/content/drive/MyDrive/AiAudio_PR_Project/AiAudio_Plot"  
18 plt.savefig(plot_path+"/real_acc_4models.png", dpi=300, bbox_inches='tight')  
19 plt.show()
```

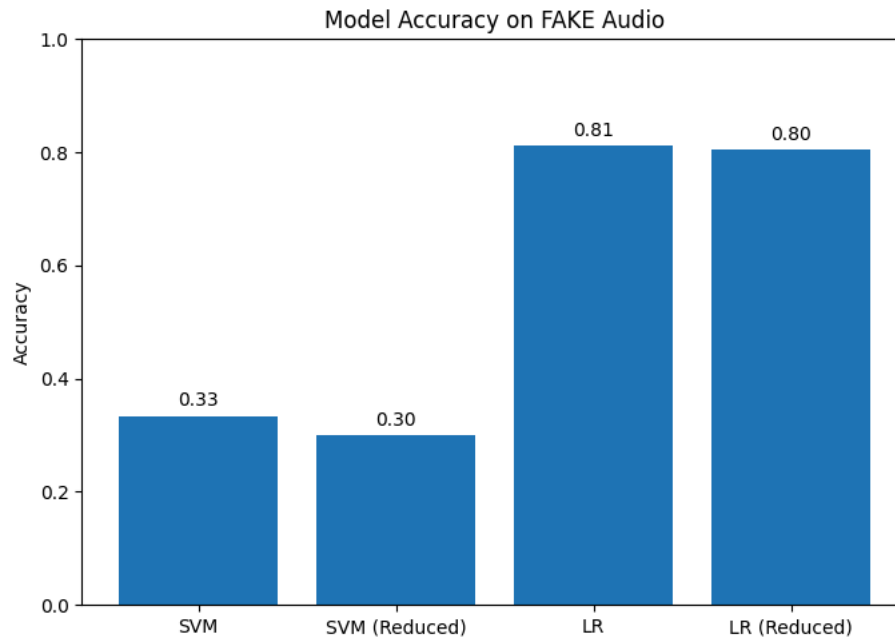


```
1  
2  
3 # -----  
4 # Plot 2: Accuracy for Fake Audio  
5 # -----  
6 plt.figure(figsize=(7, 5))  
7 bars = plt.bar(models, fake_acc)  
8 plt.ylabel("Accuracy")  
9 plt.title("Model Accuracy on FAKE Audio")  
10 plt.ylim(0, 1)
```

```

11
12 # Add values on top of bars
13 for bar in bars:
14     yval = bar.get_height()
15     plt.text(bar.get_x() + bar.get_width()/2, yval + 0.01, f"{yval:.2f}",
16             ha="center", va="bottom")
17
18
19 plt.tight_layout()
20 plot_path = "/content/drive/MyDrive/AiAudio_PR_Project/AiAudio_Plot"
21 plt.savefig(plot_path+"/fake_acc_4models.png", dpi=300, bbox_inches='tight')
22 plt.show()

```

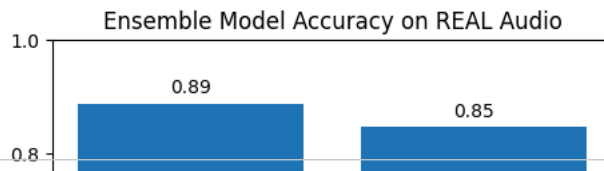


LORIS9,7 plots

```

1 # -----
2 # Plot 1: Accuracy for Real Audio
3 # -----
4 plt.figure(figsize=(5, 5))
5 bars = plt.bar(models, real_acc)
6 plt.ylabel("Accuracy")
7 plt.title("Ensemble Model Accuracy on REAL Audio")
8 plt.ylim(0, 1)
9
10 # Add values on top of bars
11 for bar in bars:
12     yval = bar.get_height()
13     plt.text(bar.get_x() + bar.get_width()/2, yval + 0.01, f"{yval:.2f}",
14             ha="center", va="bottom")
15
16 plt.tight_layout()
17 plot_path = "/content/drive/MyDrive/AiAudio_PR_Project/AiAudio_Plot"
18 plt.savefig(plot_path+"/real_acc_Ensemble_models.png", dpi=300, bbox_inches='tight')
19 plt.show()

```



```
1
2
3 # -----
4 # Plot 2: Accuracy for Fake Audio
5 # -----
6 plt.figure(figsize=(5, 5))
7 bars = plt.bar(models, fake_acc)
8 plt.ylabel("Accuracy")
9 plt.title("Ensemble Model Accuracy on FAKE Audio")
10 plt.ylim(0, 1)
11
12 # Add values on top of bars
13 for bar in bars:
14     yval = bar.get_height()
15     plt.text(bar.get_x() + bar.get_width()/2, yval + 0.01, f"{yval:.2f}",
16             ha="center", va="bottom")
17
18
19 plt.tight_layout()
20 plot_path = "/content/drive/MyDrive/AiAudio_PR_Project/AiAudio_Plot"
21 plt.savefig(plot_path+"fake_acc_Ensemble_models.png", dpi=300, bbox_inches='tight')
22 plt.show()
```

Ensemble Model Accuracy on FAKE Audio